
Constraint-Contract Multi-Agent Repair for Travel Planning

Yuxuan Shu

School of Intelligence Science and Technology, Nanjing University
Nanjing, China
yuxuanshu@smail.nju.edu.cn

Abstract

Large language models can write plausible travel itineraries, but on TravelPlanner they often fail database-grounded hard constraints such as minimum-night rules, budget limits, room requirements, and transportation consistency. We study this failure mode under our local reproduction of the official TravelPlanner evaluation protocol and introduce *Constraint-Contract Multi-Agent Repair* (CC-MAR), a verifier-mediated repair algorithm in which agents propose contract-level patches, a critic estimates cross-contract risk, and a mediator accepts only verifier-improving edits. On the 180-instance validation set, direct prompting and a stronger constraint-checklist prompt both reach 20.00% Final Pass Rate, while generic self-refinement reaches 12.22%. CC-MAR improves Final Pass Rate to 35.00%, and its ablations show small but mixed effects: removing the critic lowers performance to 33.89%, removing the board lowers it to 34.44%, but removing role specialization reaches 36.11%. A separately engineered seeded verifier-repair pipeline reaches 46.67%, and a narrower repair-heavy control reaches 48.89%. These results indicate that explicit verification and local repair are the dominant sources of reliability; contract-mediated multi-agent repair provides a more controlled and inspectable protocol, but it does not outperform benchmark-specific deterministic repair in this setting.

1 Introduction

Travel planning is an attractive testbed for language agents because success depends on more than fluent text generation. A system must combine route construction, database-grounded entity selection, cost control, and user-specific hard constraints within a single itinerary [Xie et al., 2024]. In TravelPlanner, failures such as picking an accommodation with an impossible minimum-night requirement or mixing incompatible transportation modes are fatal even when the itinerary is otherwise coherent.

Prior approaches in this repository expose the central difficulty. A direct-prompt baseline with `deepseek-v4-flash` reaches only 20.00% Final Pass Rate under our reproduced protocol. A purely programmatic planner falls to 8.89%, and a later LLM-assisted program planner reaches 12.22%, which is still below direct prompting. These numbers matter because they show that simply adding more structure does not solve the benchmark. The bottleneck is not only plan generation; it is the absence of a reliable mechanism for detecting and repairing specific constraint violations after a candidate itinerary is written.

Our starting point is a simple observation: in TravelPlanner, many hard failures are externally checkable. Accommodation minimum-night violations, unsupported transportation choices, budget overruns, and several local constraints can be audited against the benchmark database without consulting the final scorer. This changes the problem from asking one model to implicitly track every constraint into a more controlled pipeline: produce candidate plans, expose their failures with a deterministic verifier, and revise only the failing fields or route segments.

We instantiate this idea in two systems. The first is a seeded verifier-repair pipeline that combines direct and program drafts with conservative local repair. The second, *Constraint-Contract Multi-Agent Repair* (CC-MAR), is a verifier-mediated multi-agent algorithm: agents independently propose contract-specific patches, a critic reviews cross-contract risk, and a mediator accepts only patches that improve the deterministic verifier’s violation vector.

This positioning is important for the paper’s main claim. The strongest result is not produced by the most agentic system: a repair-heavy control reaches 48.89%, the seeded role pipeline reaches 46.67%, and CC-MAR reaches 35.00%. The paper therefore does not argue that adding agent roles is automatically beneficial. Instead, CC-MAR tests a narrower claim: multi-agent repair becomes scientifically meaningful only when agents communicate through explicit constraint contracts and when promotion is governed by verifier-monotone acceptance.

Our contributions are:

- We formulate TravelPlanner as a constrained generation problem in which hard feasibility can be partially externalized into deterministic verification.
- We introduce CC-MAR, a verifier-mediated multi-agent repair algorithm in which contract agents propose patch candidates and a mediator accepts only verifier-improving repairs.
- We package a reproducible local evaluation setup that preserves TravelPlanner scoring logic while redirecting validation data loading to local files in `database/validation.csv`.
- We compare direct prompting, stronger structured prompting, generic self-refinement, deterministic repair, CC-MAR, and communication/specialization ablations.
- We provide mechanism, cost, and failure analysis showing when contract-guided repair helps and which benchmark-grounded constraints remain difficult.

2 Related Work

LLM-based planning. Recent work has studied language models as planning agents that interleave reasoning and action [Yao et al., 2023b] or expand search over multiple thoughts [Yao et al., 2023a]. These methods show that decomposition can help difficult decision-making tasks, but they do not by themselves guarantee database-grounded constraint satisfaction. Our setting differs in that many failures are externally checkable, so verification is not a formatting choice but the main design variable.

Self-refinement and critique. Self-refinement methods use iterative feedback to improve model outputs [Madaan et al., 2023, Shinn et al., 2023]. This line of work motivates our multi-round design, but generic self-critique is often too weak for benchmark constraints that depend on a fixed entity database. In our system, critique is grounded by deterministic audit results, so the reviser receives explicit failure types instead of free-form advice.

Program-aided reasoning and verification. Program-aided LLM methods offload parts of reasoning to external code [Gao et al., 2022], while decomposition-style prompting methods separate planning from execution [Wang et al., 2023, 2022]. Our work is closest in spirit to these approaches: instead of trusting a single forward pass, we use programmatic checks to expose failure modes that the model frequently misses. The difference is that we study which externalized failures matter most on a multi-constraint travel-planning benchmark with end-to-end constraint evaluation.

Benchmarking and LLM-based evaluation. TravelPlanner provides a realistic planning benchmark with structured reference information and constraint-aware evaluation [Xie et al., 2024]. We use the benchmark scorer only for final reporting, not for per-sample selection. For internal judgment, we draw on the broader idea that LLM-based evaluation can be useful when paired with structured criteria [Liu et al., 2023], but in our pipeline deterministic checks remain authoritative on all database-grounded constraints and LLM judgment is only a secondary signal.

3 Problem Formulation

Each TravelPlanner instance consists of a user request q , structured reference information E , and a set of benchmark constraints. The request specifies an origin city, destination region, trip length, number of travelers, budget, and optional local preferences such as cuisine or room type. The reference information provides candidate restaurants, attractions, accommodations, and transportation options extracted from the benchmark database.

The goal is to generate a day-level itinerary

$$y = (d_1, d_2, \dots, d_T),$$

where each day d_t contains the current city, transportation, breakfast, attraction(s), lunch, dinner, and accommodation. Let C_h denote hard constraints such as budget, valid entities, route closure, visiting-city count, transportation validity, minimum-night rules, and user-specified local constraints. Let C_s denote softer quality criteria such as diversity or stylistic naturalness. A plan is *feasible* if it satisfies all constraints in C_h .

Our objective is not to solve a global optimization problem exactly. Instead, we approximate

$$\max_y U(y, q, E) \quad \text{subject to } y \text{ satisfies } C_h,$$

where U captures itinerary quality, while the benchmark scorer provides the final binary notion of success. In practice, the paper focuses on Final Pass Rate, the most stringent reproduced benchmark metric, and also reports commonsense and hard-constraint pass rates for diagnostic analysis.

4 Method

4.1 Overview

Figure 1 shows the seeded verifier-repair pipeline used in the earlier control experiments. The central design choice is to separate generation from verification. Rather than expecting one model call to track every benchmark constraint internally, we generate candidate itineraries, expose their failures through deterministic checks, and revise only the parts that the verifier marks as unsafe.

The final method, CC-MAR, makes this separation more explicit. It treats each checkable requirement as a *constraint contract*. Contract agents independently propose patches for different failure classes, a critic estimates whether each patch may break another contract, and a deterministic mediator accepts only patches that improve the verifier’s violation vector. This design gives the multi-agent system a concrete communication protocol and acceptance rule rather than a sequence of loosely connected prompts. We treat the particular role partition as an implementation choice and test it directly in ablations.

4.2 Seeder Agent

The seeder agent loads two heterogeneous drafts from local artifacts:

- a direct-prompt itinerary from `outputs/validation`;
- a program-generated itinerary from `outputs_program_v23_best/validation`.

These drafts provide complementary strengths. The direct seed is usually stronger on overall completeness, while the program seed is often closer to database-supported transportation and entity choices.

When the seed pair is ambiguous or low-confidence, the planner agent additionally generates a scratch draft in structured JSON. In the final implementation, this scratch branch is activated adaptively: if the seed pair is already high-confidence under deterministic audit, the system skips scratch generation to save cost without changing the selection rule.

4.3 Planner Agent

The planner agent produces a structured itinerary rather than free-form prose. Its prompt explicitly asks for a route-first plan, exact day count, and concrete visited cities drawn from the reference candidates. This structured output reduces parse failures and makes downstream repair more local.

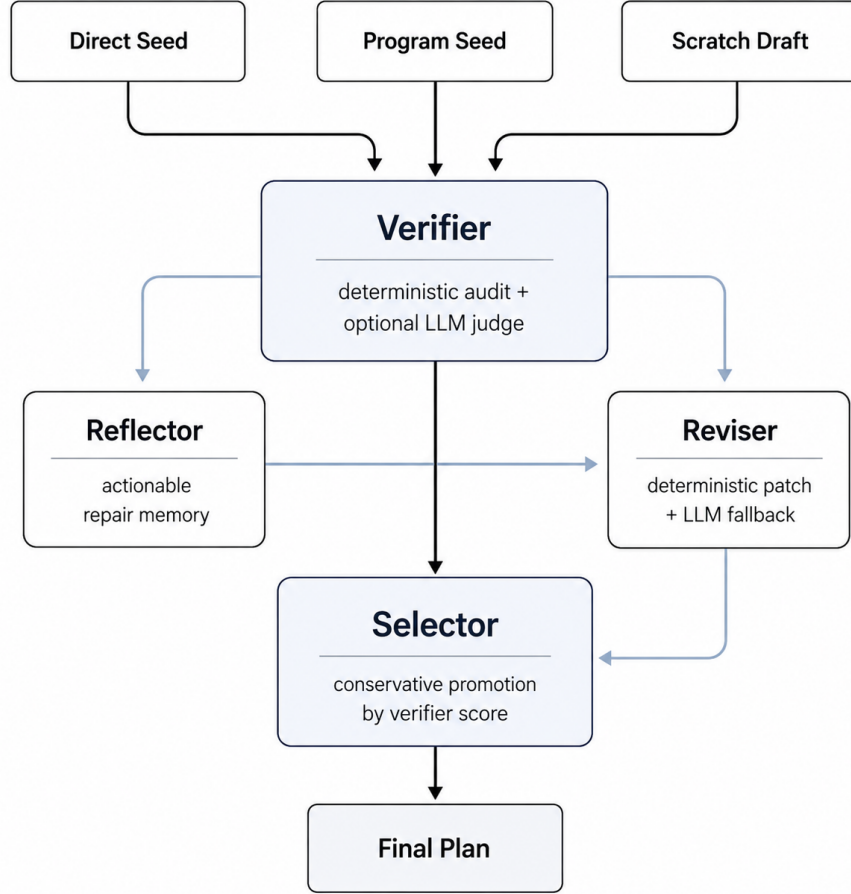


Figure 1: Overview of the planning pipeline. Multiple seed plans are audited by a verifier, repaired through reflection and revision, and conservatively promoted by a selector before producing the final plan. Dark arrows mark the main path from seeding to final selection, while lighter arrows show auxiliary repair flows.

In the final system, the scratch planner is not claimed as the main driver of improvement. It acts as a diversity source on ambiguous cases. This distinction matters empirically: on many validation instances, the highest-leverage improvement comes from revising a flawed seed rather than inventing a fully new itinerary.

4.4 Verifier Agent

The verifier is the core reliability component. It combines deterministic benchmark-grounded checks with optional LLM judgments for tie-breaking and repair guidance. Deterministic checks are authoritative for:

- route closure and visiting-city count;
- transportation validity and mode consistency;
- entity validity for restaurants, attractions, and accommodations;
- accommodation minimum-night rules;
- budget estimation;
- user local constraints such as room type, house rule, and cuisine.

The verifier assigns each candidate an audit score, a likely-pass flag, fatal issues, and warnings. Crucially, the final TravelPlanner evaluator is never used for per-sample selection.

4.5 Reflector and Reviser Agents

The reflector converts verifier outputs into actionable memory. For deterministically diagnosed errors such as minimum-night violations or budget overruns, the reflection can be generated directly from the audit signal. For more ambiguous cases, an LLM reflector summarizes concrete repairs instead of generic advice.

The reviser applies *local repair*. It first attempts deterministic patches, with emphasis on the high-frequency failure modes observed in the repository:

- replacing illegal accommodations with valid alternatives in the same city;
- inserting cuisine-compliant restaurants when required;
- reducing cost via cheaper valid meals or lodging.

If deterministic repair does not resolve the issue, the system falls back to an LLM reviser conditioned on the verifier evidence. Revised candidates are promoted only if they are conservatively better than the best seed under deterministic audit.

4.6 Failure Handling and Reproducibility

The full pipeline is resume-safe. All agent calls are cached, each instance writes an incremental debug file, and interrupted runs can continue with `RESUME=1`. This design is important because long API runs occasionally fail with transient network errors; the system is built to preserve partial progress rather than recomputing from scratch.

4.7 Algorithmic Summary

For each instance, the pipeline executes the following steps:

1. Load the direct and program seed drafts, audit them deterministically, and rank them with a verifier-first score.
2. Generate a scratch draft only when the seed pair is not already high-confidence.
3. Apply deterministic verification to all round-1 candidates and call the optional LLM verifier only on selected candidates that are ambiguous, low-confidence, or newly generated.
4. Choose repair targets from the verified pool, build reflection memory from explicit audit failures, and apply deterministic local repair before any LLM fallback.
5. Re-verify revised candidates and promote them only when they are conservatively stronger than the best seed under deterministic audit.
6. Let the selector break close ties among already promotable candidates, then emit the final plan and a per-instance debug record.

4.8 Constraint-Contract Multi-Agent Repair

CC-MAR formalizes repair as contract-level patch arbitration. A contract $k \in K$ is a checkable predicate or partial predicate over a task q , environment E , and plan y . In TravelPlanner, contracts include route closure, city count, sandbox entity validity, transportation consistency, accommodation legality, budget, and user local constraints.

Given the deterministic verifier V , we summarize a plan by a violation vector

$$v(y) = (\text{fatal}, \text{invalid entities}, \text{invalid transport}, \text{min-night misses}, \\ \text{local misses}, \text{budget miss}, \text{warnings}, -\text{score}).$$

The mediator compares vectors lexicographically, where smaller is better. A patch p is *contract-dominating* if $v(p(y)) <_{\text{lex}} v(y)$ and no protected hard contract changes from pass to fail.

Our default CC-MAR implementation uses five roles. The route agent proposes route and transportation repairs; the entity agent repairs sandbox and current-city violations; the lodging agent repairs minimum-night, room-rule, and room-type failures; the budget agent proposes cost and cuisine repairs; and the critic agent reviews proposed patches for cross-contract risk. Agents communicate only

through a shared violation and patch board. The mediator is verifier-first: it re-audits each proposed patch and accepts at most one contract-dominating patch per round. Because role specialization is not assumed to be beneficial, we also evaluate no-specialization and single-agent variants.

Proposition. Under a deterministic verifier and finite patch budget, a CC-MAR run that accepts only contract-dominating patches produces a sequence of plans whose violation vectors monotonically improve under V and terminates after finitely many accepted patches.

Proof sketch. Each accepted patch strictly decreases a finite lexicographic vector and is forbidden from regressing protected hard contracts. Because each round contains a finite number of proposed patches and the run has a finite round budget, the process cannot cycle. This guarantee is internal to the verifier; it does not guarantee reproduced benchmark validity or real-world travel feasibility.

5 Experimental Setup

Dataset. All experiments use the locally stored TravelPlanner validation split. The split contains 180 instances in `sole-planning` mode, balanced across difficulty levels and trip lengths.

Baselines. We compare against existing repository baselines, stronger prompt baselines, and repair controls:

- **Direct Prompt:** a one-shot LLM itinerary generator;
- **Constraint-Checklist Direct:** a stronger structured direct prompt with explicit checklist constraints and JSON output;
- **Generic Self-Refine:** a prompt-only draft–critique–revise baseline without deterministic repair or verifier-based selection;
- **Program Planner v1:** a rule-based planner using extracted entities;
- **LLM-Assisted Program Planner:** the stored `program_v23_best` artifact;
- **Verifier-Guided Hybrid Selector:** a previous control that selects between direct and program drafts without structured multi-round repair.

We also report a **Verifier-Directed Repair Control**, the earlier seeded verifier-repair pipeline, and **CC-MAR**. For CC-MAR, we include ablations that remove the critic agent, remove the shared communication board, remove role specialization, or replace the specialist team with a single generic repair agent.

Model and decoding. All LLM-based components use `deepseek-v4-flash`. Structured direct generation uses temperature 0.1, generic self-refinement uses temperature 0.1 for drafting and 0.05 for revision, and CC-MAR agents use temperature 0.05 with a deterministic mediator. The earlier seeded pipeline uses two rounds, up to three candidates, and the default verifier-gated repair settings in `seeded_multi_agent_planner.py`; CC-MAR uses three repair rounds unless otherwise stated.

Metrics. We report the TravelPlanner metrics produced by our local reproduction of the official evaluation protocol: Delivery Rate, Commonsense Constraint Micro/Macro Pass Rate, Hard Constraint Micro/Macro Pass Rate, and Final Pass Rate. Final Pass Rate is the primary metric.

Local evaluation pipeline. To make experiments reproducible in this environment, we preserve the original scoring logic while redirecting validation data loading to local files through `utils/local_data.py` and `evaluation/run_local_eval.py`. We refer to this setup as *our local reproduction of the official TravelPlanner evaluation protocol*. The final scorer is used only after a complete submission file is produced. It is never used for per-instance candidate selection, which relies only on internal verifier scores and deterministic repair checks. All headline numbers are artifact-level outcomes from single reproduced validation runs; we do not report repeated randomized trials or confidence intervals. For internal auditing, a missing-accommodation marker on the final return day is treated as valid when the itinerary closes back at the origin, but all headline numbers in the paper are determined by the reproduced benchmark protocol rather than the internal audit alone.

Table 1: Main validation results under our local reproduction of the official TravelPlanner evaluation protocol. All entries are reproduced benchmark pass rates (% , higher is better). The repair-heavy control remains strongest overall, while CC-MAR provides a contract-mediated multi-agent repair protocol that improves over prompt-only and generic self-refinement baselines.

Method	Final Pass (%) ↑	Common. Macro (%) ↑	Hard Macro (%) ↑	Common. Micro (%) ↑	Hard Micro (%) ↑
<i>Prompt-only and program baselines</i>					
Direct Prompt	20.00	20.56	57.22	80.62	60.95
Program Planner v1	8.89	21.11	16.67	71.67	24.05
LLM-Assisted Program Planner	12.22	18.33	42.78	82.15	35.48
<i>Stronger prompting baselines</i>					
Constraint-Checklist Direct	20.00	20.56	57.22	77.43	60.95
Generic Self-Refine	12.22	13.33	51.67	74.17	57.62
<i>Verifier and repair controls</i>					
Verifier-Guided Hybrid Selector	27.22	31.11	65.00	87.22	63.81
Verifier-Directed Repair Control	48.89	52.78	<u>68.89</u>	90.90	<u>65.71</u>
<i>Seeded pipeline</i>					
Seeded Multi-Agent Planner	<u>46.67</u>	<u>49.44</u>	67.22	<u>90.07</u>	61.67
<i>Contract multi-agent repair</i>					
CC-MAR	35.00	38.33	72.22	88.26	71.19

Note: Best values are bolded and second-best values are underlined.

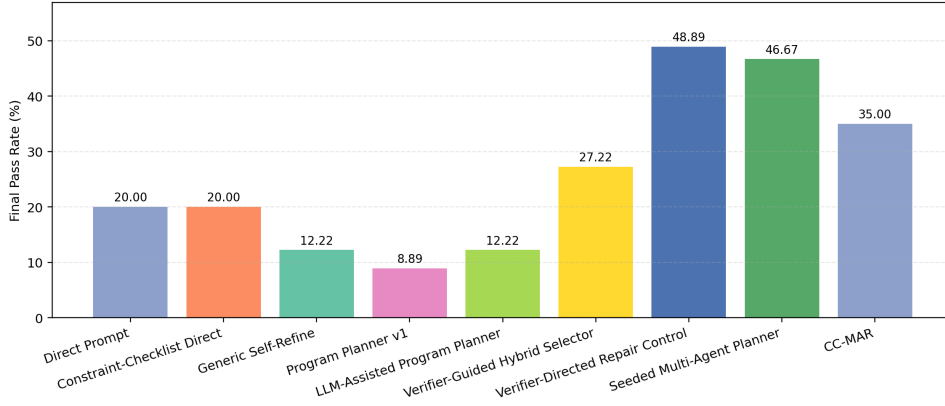


Figure 2: Final Pass Rate across reproduced baselines, controls, and repair methods. The large gap between prompt-only baselines and verifier-repair methods indicates that feasibility, not language fluency, is the main bottleneck on TravelPlanner.

Research questions. We organize the results around five questions: **RQ1** Does verifier-directed repair improve final validity over direct prompting and stronger prompt baselines? **RQ2** Does the gain come mainly from deterministic local repair or from multi-agent contract arbitration? **RQ3** Which CC-MAR components matter? **RQ4** How does the full pipeline use extra LLM budget in practice? **RQ5** Which benchmark constraints remain the main sources of failure?

6 Results

RQ1: Does verifier-directed repair improve final validity over direct prompting and earlier repository baselines? Yes. Table 1 and Figure 2 show that validity improves materially only once the system can expose benchmark-grounded failures and repair them locally. Direct prompting reaches 20.00% Final Pass Rate, and the constraint-checklist direct prompt also reaches 20.00%, indicating that a stronger output schema alone does not address the main bottleneck. Generic self-refinement performs worse at 12.22%, which suggests that unconstrained critique can damage already valid fields. Verifier-based seed selection raises Final Pass Rate to 27.22%, CC-MAR reaches 35.00%, the full seeded pipeline reaches 46.67%, and the repair-heavy control reaches 48.89%.

The results separate two claims. First, verifier-directed local repair is the dominant empirical mechanism: the repair-heavy control improves Final Pass Rate by 28.89 points over direct prompting.

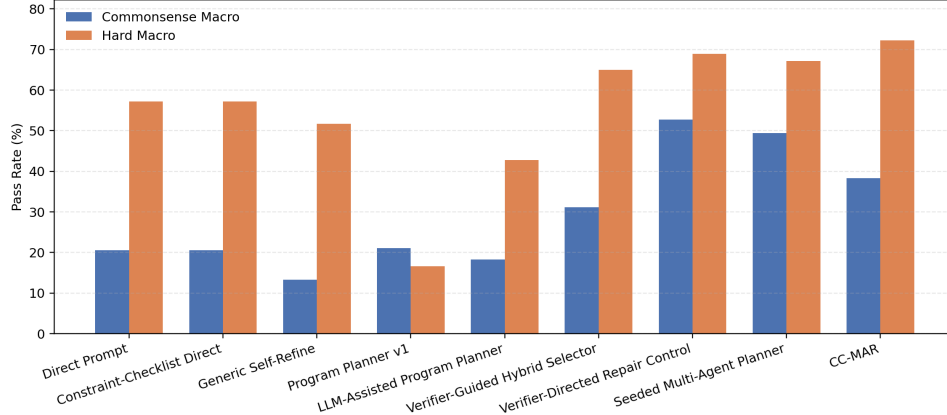


Figure 3: Macro-level commonsense and hard-constraint pass rates. Repair-based methods improve both axes simultaneously, suggesting that gains are not limited to one narrow subset of constraints.

Table 2: Progressive ablation of the main components. Verifier-based seed selection helps, but most of the gain comes from targeted repair. The full seeded loop remains much stronger than direct prompting even though the narrower repair control is slightly higher. All metrics are macro or final pass rates (%), higher is better).

Variant	Final Pass (%) ↑	Δ Final vs. prev.	Common. Macro (%) ↑	Hard Macro (%) ↑
Direct Prompt	20.00	—	20.56	57.22
+ Verifier-Based Seed Selection	27.22	+7.22	31.11	65.00
+ Deterministic Repair	48.89	+21.67	52.78	68.89
+ Full Seeded Multi-Agent Loop	46.67	-2.22	49.44	67.22

Second, CC-MAR provides a more explicit multi-agent repair protocol, but its 35.00% Final Pass Rate remains below the more benchmark-specific seeded repair systems. The central empirical conclusion is therefore not that more agents are inherently better; it is that hard-constraint exposure and conservative patch acceptance matter more than unconstrained generation or generic self-critique.

7 Analysis and Ablations

7.1 RQ2: Where does the gain come from?

Most of the gain comes from repair, not from additional agent freedom. Table 2 shows that verifier-based seed selection improves Final Pass Rate from 20.00% to 27.22%, so heterogeneous drafts already contain useful complementary signal. The largest jump occurs when deterministic repair is added: Final Pass Rate rises by another 21.67 points to 48.89%. The full seeded multi-agent loop then reaches 46.67%, slightly below the repair-heavy control but still far above both direct prompting and seed selection alone.

This gap narrows the claim the paper can make. The results do not support the statement that freer multi-agent interaction is inherently stronger than a tighter repair policy on TravelPlanner. They support a more specific conclusion: seed diversity plus verifier-directed repair produces a much stronger planner than prompt-only generation, and the dominant mechanism is local correction guided by explicit failure exposure.

7.2 RQ3: Which CC-MAR components matter?

Table 3 shows mixed evidence for the multi-agent components. Removing the critic lowers Final Pass Rate from 35.00% to 33.89%, and removing the shared board lowers it to 34.44%, which is consistent with the hypothesis that cross-contract review and shared state can prevent some unsafe repairs. However, removing role specialization increases Final Pass Rate to 36.11%, and a single generic repair agent reaches 33.89%. These results do not support a strong claim that the chosen specialist decomposition is optimal. They support a narrower and more defensible claim: verifier-

Table 3: CC-MAR component ablations. The critic and shared board provide small gains over their removals, but removing role specialization performs slightly better than the specialized-agent configuration. This negative result suggests that the current contract protocol is useful as controlled patch arbitration, while the particular role partition is not yet optimized.

Variant	Final Pass (%) $\uparrow$	Common. Macro (%) $\uparrow$	Hard Macro (%) $\uparrow$	Hard Micro (%) $\uparrow$
CC-MAR	35.00	38.33	72.22	71.19
w/o Critic Agent	33.89	37.22	71.67	69.52
w/o Communication Board	34.44	39.44	70.00	69.52
w/o Role Specialization	36.11	40.56	71.11	70.48
Single Generic Repair Agent	33.89	37.78	71.11	70.48

Table 4: Behavior and cost profile of the seeded multi-agent method on all 180 validation instances. Incremental LLM calls count scratch planning, first-round LLM verification, LLM reflection, and LLM revision beyond the two stored seed drafts. The profile shows that the system usually repairs or selects existing seeds rather than regenerating plans from scratch.

Statistic	Count / 180	Share / Value
<i>Selection behavior</i>		
Final revised candidate selected	101	56.1%
Original seed selected	79	43.9%
<i>Adaptive generation</i>		
Scratch planner generated	68	37.8%
Scratch planner skipped	112	62.2%
<i>Reflection mode</i>		
Deterministic reflection	133	73.9%
LLM reflection	47	26.1%
<i>LLM budget</i>		
Average incremental LLM calls / instance	2.21	
Maximum incremental LLM calls	6	

mediated patch arbitration improves over prompt-only baselines, while the best way to allocate repair responsibilities across agents remains open.

7.3 Round-2 revision still matters inside the full pipeline

On the diagnostic medium slice (instances 21–40), round-2 revision increases the best verifier score on 12 of 20 instances (60%). The final selected plan is a revised candidate in 15 of 20 cases. This is important because it rules out a trivial explanation in which the system merely re-ranks pre-existing seeds.

The distribution of final choices on this slice is:

- `program_seed_revised_r2`: 14
- `program_seed`: 2
- `direct_seed`: 3
- `direct_seed_revised_r2`: 1

This pattern suggests that the program seed provides a strong structural scaffold, while revision corrects the specific hard failures that prevent it from passing the reproduced benchmark protocol.

7.4 RQ4: How does the full pipeline use extra LLM budget?

Table 4 and Figure 4 show that the method behaves conservatively rather than rewriting everything. A revised candidate is selected in 101 of 180 instances, while an original seed remains best in 79 cases. The most common final choice is `program_seed_revised_r2` (80 instances), followed by `direct_seed` (44), `program_seed` (35), and `direct_seed_revised_r2` (21). The scratch planner is invoked on only 68 instances and skipped on 112 high-confidence seed pairs, which keeps the average incremental LLM budget at 2.21 calls per instance. This supports the intended design: use seed diversity and explicit repair first, and reserve fresh generation for ambiguous cases.

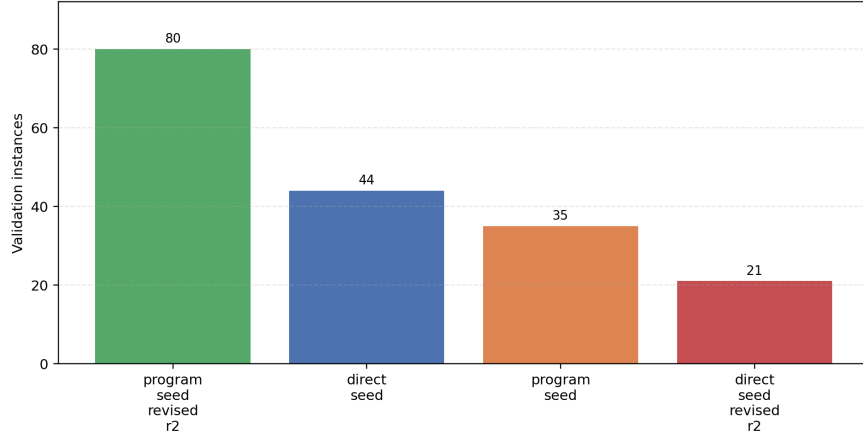


Figure 4: Distribution of final selections for the seeded multi-agent planner. Most wins come from repairing the program seed, while direct and program seeds still survive unchanged on a substantial minority of instances.

Table 5: Remaining failure taxonomy for the seeded multi-agent method under our reproduced evaluation protocol. Counts aggregate criterion-level false checks from the benchmark detailed scores, so a single plan may contribute to multiple rows.

Failure category	False checks	Share of all false checks (%)
Diverse Restaurants	36	22.64
Non-conf. Transportation	34	21.38
Complete Information	34	21.38
Minimum Nights Stay	25	15.72
Within Sandbox	10	6.29
Room Type	6	3.77
Room Rule	5	3.14
Budget	4	2.52

7.5 What the Reviser Actually Fixes

The strongest internal evidence comes from the repair logs. Under deterministic audit, successful direct/program repairs are dominated by accommodation substitution. Across the full validation set, deterministic repair produces 57 additional audit-clean plans beyond the original seed union, and 82 successful repairs include an accommodation patch. Budget-reduction patches appear much less frequently, and cuisine-driven patches are rare.

This asymmetry is informative rather than incidental. It indicates that the current benchmark state is dominated by a narrow class of high-impact hard failures: seed plans often choose otherwise plausible accommodations that violate minimum-night rules or local lodging constraints. These failures are easy to miss in end-to-end prompting but easy to repair once surfaced explicitly.

7.6 RQ5: Which constraints remain difficult?

Table 5 shows that the remaining errors are concentrated rather than diffuse. The most common false checks involve restaurant diversity, transportation consistency, complete-information checks, and minimum-night constraints. This pattern is consistent with the benchmark’s structure: once budget and room-rule violations are repaired, the hardest residual cases are those that require coordinated day-level coverage across multiple fields rather than a single local substitution.

Table 6: Final Pass Rate by difficulty level (% , higher is better). Repair-based methods remain stronger than direct prompting across all difficulty groups, with the largest gains on easy and medium instances.

Method	Easy (%) ↑	Medium (%) ↑	Hard (%) ↑
Direct Prompt	26.67	11.67	21.67
Constraint-Checklist Direct	26.67	11.67	21.67
Generic Self-Refine	20.00	1.67	15.00
Program Planner v1	15.00	10.00	1.67
LLM-Assisted Program Planner	18.33	15.00	3.33
Verifier-Guided Hybrid Selector	35.00	23.33	23.33
Verifier-Directed Repair Control	78.33	40.00	28.33
Seeded Multi-Agent Planner	76.67	41.67	21.67
CC-MAR	48.33	28.33	28.33

Table 7: Final Pass Rate by trip length (% , higher is better). Long-horizon itineraries remain harder, but repair-based methods reduce the gap substantially on 5-day and 7-day trips.

Method	3 days (%) ↑	5 days (%) ↑	7 days (%) ↑
Direct Prompt	40.00	13.33	6.67
Constraint-Checklist Direct	40.00	13.33	6.67
Generic Self-Refine	21.67	10.00	5.00
Program Planner v1	26.67	0.00	0.00
LLM-Assisted Program Planner	28.33	5.00	3.33
Verifier-Guided Hybrid Selector	53.33	18.33	10.00
Verifier-Directed Repair Control	61.67	45.00	40.00
Seeded Multi-Agent Planner	61.67	40.00	38.33
CC-MAR	55.00	30.00	20.00

7.7 Breakdown by Trip Length and Difficulty

We further break down final-pass performance by difficulty and trip length in Tables 6 and 7. The broad pattern is that repair-based methods remain stronger across all groups. For the full seeded multi-agent method, the improvement over direct prompting is especially large on 5-day trips (40.00% vs. 13.33%) and 7-day trips (38.33% vs. 6.67%), while the repair-heavy control remains the strongest method on the hardest long-horizon settings.

7.8 Case Studies

Table 8 compresses three representative trajectories into a common format. The first two are successful because the pipeline preserves a strong route scaffold and patches a small number of high-impact fields. The third remains a failure because transportation legality is less local than accommodation replacement and can still break after otherwise sensible edits.

8 Limitations

The paper has several limitations.

First, the strongest full-validation result comes from a repair-heavy control (48.89%) rather than from either the seeded role pipeline (46.67%) or CC-MAR (35.00%). This narrows the interpretation of our contribution: the main gain comes from explicit verifier-guided correction, while contract-mediated multi-agent repair adds a cleaner arbitration protocol and better diagnostics but not the absolute best score.

Second, the deterministic verifier and local repair rules are benchmark-specific. Although the checks are transparent and reproducible, they are written against the TravelPlanner database schema and cannot be assumed to transfer unchanged to other planning environments.

Third, the internal audit and the reproduced benchmark scorer are related but not identical. We avoid using the final scorer for per-sample selection, but this also means that some high-audit candidates

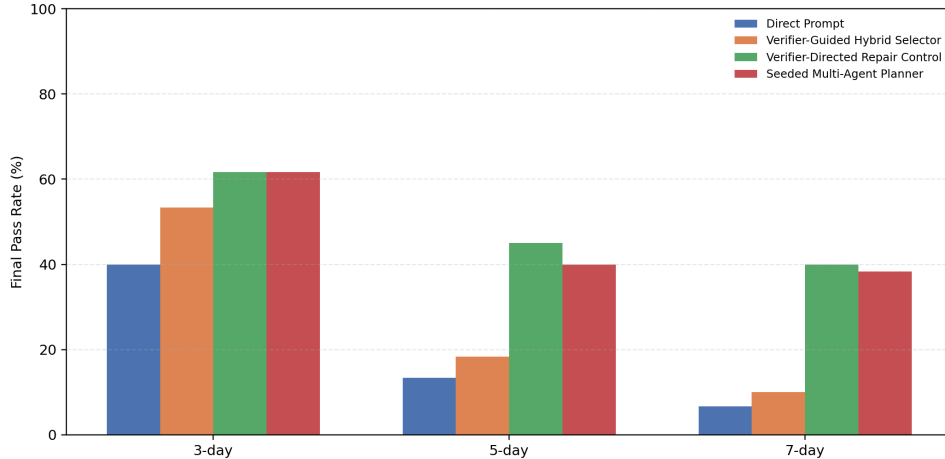


Figure 5: Final Pass Rate by trip length for representative methods. Repair-based methods are especially valuable on 5-day and 7-day instances, where route continuity and accommodation legality interact more often.

Table 8: Representative repair trajectories. Successful cases usually preserve an existing scaffold and repair a small number of benchmark-critical fields, while the remaining failures involve less local transportation constraints.

Case	Seed failure	Repair action	Outcome
21	Program seed uses an Orlando accommodation with a 30-night minimum stay.	Replace the lodging with a valid short-stay option and slightly reduce meal cost.	Converts a one-fatal plan into a likely-pass candidate under reproduced evaluation.
28	Direct seed is incomplete and the program seed contains an invalid Norfolk accommodation.	Preserve the stronger route scaffold and patch the lodging locally.	Produces a complete five-day plan with verifier score 100 on the diagnostic run.
32	Revised candidate still introduces an invalid flight despite otherwise plausible daily structure.	Attempt local transportation repair, then fall back to the LLM reviser when deterministic patching is insufficient.	Remains a benchmark failure, illustrating brittle route repair when the valid transport space is sparse.

still fail the reproduced benchmark metric. The paper therefore distinguishes internal audit analysis from final benchmark results whenever the two are compared.

Fourth, the CC-MAR ablations show that the current role specialization is not reliably beneficial: the no-specialization variant reaches 36.11%, slightly above the specialized CC-MAR configuration. This does not invalidate verifier-mediated patch arbitration, but it does mean that the role design should be treated as an implementation choice rather than a settled algorithmic optimum.

Fifth, the paper includes a generic self-refinement baseline, but it does not yet include Reflexion-style memory baselines, multi-model replication on other frontier LLMs, or repeated randomized trials. We therefore position the paper as a benchmark-grounded study of one reproduced setting rather than a broad claim of model-agnostic superiority.

Sixth, API-based experiments remain sensitive to runtime and networking noise. The system is resume-safe, but cost and wall-clock latency are still higher than single-pass prompting, and the scratch branch has not been isolated as an independent source of uplift beyond its role as an adaptive diversity mechanism.

9 Ethics and Societal Impact

This work studies benchmarked travel planning rather than deployment to real users. Even so, constraint-violating travel plans can be misleading or unsafe if used without verification. A language model may recommend infeasible transportation, illegal lodging arrangements, or budget-breaking itineraries. Our method reduces some of these risks by externalizing checkable constraints, but it does not eliminate the possibility of unsafe recommendations.

The benchmark itself is static and database-bounded. It does not model real-time availability, safety advisories, accessibility needs, or social bias in recommendations. As a result, a plan that passes benchmark constraints should not be interpreted as real-world travel advice. The main ethical value of this work is methodological: it shows how to make LLM planning more auditable in settings where external constraints are known.

10 Conclusion

This paper studies TravelPlanner through the lens of constraint exposure and local repair. Under our local reproduction of the official TravelPlanner evaluation protocol, CC-MAR improves Final Pass Rate from 20.00% direct prompting to 35.00%, while the full seeded verifier-repair pipeline reaches 46.67% and the stronger repair-heavy control reaches 48.89%. The clearest scientific conclusion is therefore not that multi-agent freedom is universally best, but that verifier-directed local repair is the dominant mechanism behind reliable benchmark performance in this setting.

CC-MAR remains useful because it turns multi-agent repair into an auditable protocol: agents propose patches, a critic records cross-contract risk, and a mediator accepts only verifier-improving edits. The fact that a no-specialization ablation slightly outperforms the specialized version is also informative: future progress may depend less on adding more named roles and more on tighter interfaces between generation, verification, arbitration, and local correction.

References

- Luyu Gao, Shouyuan Chen, Jiachen Zhang, Ruilin Shao, and Jun Wang. Pal: Program-aided language models. *arXiv preprint arXiv:2211.10435*, 2022.
- Yang Liu, Dan Iter, Yichong Xu, Shuhang Wang, Ruochen Xu, and Chenguang Zhu. G-eval: Nlg evaluation using GPT-4 with better human alignment. *arXiv preprint arXiv:2303.16634*, 2023.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Kavya Gupta, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. *arXiv preprint arXiv:2303.17651*, 2023.
- Noah Shinn, Federica Cassano, Edward Berman, Ashwin Gopinath, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*, 2023.
- Lei Wang, Wanyu Xu, Yihui Lan, Zhiqiang Hu, Roy Ka-Wei Lee, and Ee-Peng Lim. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. *arXiv preprint arXiv:2305.04091*, 2023.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022.
- Jian Xie, Kai Zhang, Jiangjie Chen, Tinghui Zhu, Renze Lou, Yuandong Tian, Yanghua Xiao, and Yu Su. Travelplanner: A benchmark for real-world planning with language agents. In *Forty-first International Conference on Machine Learning*, 2024.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *arXiv preprint arXiv:2305.10601*, 2023a.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2023b.

A Implementation and Reproducibility Details

A.1 Algorithmic and modeling scope

Role-specialized multi-agent scope. The paper uses “multi-agent” in an operational sense: agents have distinct prompts, explicit input/output contracts, a shared board, critic review, and mediator-controlled promotion, but they share the same base model. We do not claim decentralized learning, strategic negotiation, or emergent cooperation.

Environment. All experiments were run from the local repository root using the project virtual environment and locally stored benchmark data.

Outputs. The multi-agent runner writes generation outputs, incremental debug files, and cached agent calls to dedicated strategy-specific directories. This prevents overwriting older baselines and enables interrupted experiments to resume safely.

Evaluation. Final metrics are produced by `evaluation/run_local_eval.py`, which wraps our local reproduction of the official TravelPlanner evaluation protocol. The final scorer is never used for per-sample candidate selection.

A.2 Exact command templates

The following command templates reproduce the main paper artifacts discussed in the text:

Main seeded pipeline generation.

- `source .venv/bin/activate`
- `MODEL_NAME=deepseek-v4-flash`
- `STRATEGY=multi_agent_seeded_r2_n3`
- `OUT_ROOT=outputs_multi_agent_seeded_r2_n3`
- `python seeded_multi_agent_planner.py`

Evaluation of the main seeded submission.

- `python evaluation/run_local_eval.py`
- `-set_type validation`
- `-evaluation_file_path evaluation/validation_deepseek-v4-flash_multi_agent_seeded_r2_n3_sole-planning_submission.jsonl`
- `-output_json evaluation/experiment_results/multi_agent_seeded_r2_n3_metrics_local.json`

Evaluation of the repair control submission.

- `python evaluation/run_local_eval.py`
- `-set_type validation`
- `-evaluation_file_path evaluation/validation_deepseek-v4-flash_seeded_repair_probe_sole-planning_submission.jsonl`
- `-output_json evaluation/experiment_results/seeded_repair_probe_metrics_local.json`
- `python scripts/build_report_artifacts.py`

Strong prompt and CC-MAR experiments.

- `MODEL_NAME=deepseek-v4-flash STRATEGY=constraint_direct_json python strong_baseline_runner.py`
- `MODEL_NAME=deepseek-v4-flash STRATEGY=generic_self_refine_r1 python strong_baseline_runner.py`
- `MODEL_NAME=deepseek-v4-flash STRATEGY=cc_mar_r3 ROUNDS=3 python contract_multi_agent_repair.py`
- `MODEL_NAME=deepseek-v4-flash STRATEGY=cc_mar_no_critic ROUNDS=3 DISABLE_CRITIC=1 python contract_multi_agent_repair.py`
- `MODEL_NAME=deepseek-v4-flash STRATEGY=cc_mar_no_board ROUNDS=3 DISABLE_BOARD=1 python contract_multi_agent_repair.py`
- `MODEL_NAME=deepseek-v4-flash STRATEGY=cc_mar_no_specialization ROUNDS=3 GENERIC_AGENTS=1 python contract_multi_agent_repair.py`
- `MODEL_NAME=deepseek-v4-flash STRATEGY=single_generic_repair_agent ROUNDS=3 SINGLE_AGENT=1 python contract_multi_agent_repair.py`

These templates map one-to-one from generation output or stored submission file, to reproduced evaluation JSON, to the paper tables and figures generated in Styles/assets.

A.3 Artifact pointers

The main paper claims can be traced to the following files:

- Main seeded pipeline metrics: `evaluation/experiment_results/multi_agent_seeded_r2_n3_metrics_local.json`
- Repair control metrics: `evaluation/experiment_results/seeded_repair_probe_metrics_local.json`
- Constraint-checklist direct metrics: `evaluation/experiment_results/constraint_direct_json_metrics_local.json`
- Generic self-refine metrics: `evaluation/experiment_results/generic_self_refine_r1_metrics_local.json`
- CC-MAR metrics: `evaluation/experiment_results/cc_mar_r3_metrics_local.json`
- CC-MAR ablation metrics: `evaluation/experiment_results/cc_mar_no_critic_metrics_local.json`, `evaluation/experiment_results/cc_mar_no_board_metrics_local.json`, `evaluation/experiment_results/cc_mar_no_specialization_metrics_local.json`, and `evaluation/experiment_results/single_generic_repair_agent_metrics_local.json`
- Aggregated paper summary: `evaluation/experiment_results/paper_artifact_summary.json`
- Multi-agent profile summary: `evaluation/experiment_results/multi_agent_profile_summary.json`
- Main submission file: `evaluation/validation_deepseek-v4-flash_multi_agent_seeded_r2_n3_sole-planning_submission.jsonl`
- Repair control submission file: `evaluation/validation_deepseek-v4-flash_seeded_repair_probe_sole-planning_submission.jsonl`
- Per-instance multi-agent debug logs: `outputs_multi_agent_seeded_r2_n3/debug/debug_*.json`
- Per-instance CC-MAR debug logs: `outputs_cc_mar_r3/debug/debug_*.json`

A.4 Boundary between internal audit and benchmark validity

The internal audit uses benchmark-grounded deterministic checks to guide selection and repair. When a trip returns to the origin on the final day, the internal prompts and audit allow `Accommodation:` – on that return day. Final validity, however, is always determined by the reproduced `TravelPlanner` evaluation protocol after the complete submission file is produced.

B Reproducibility Checklist

- Code, prompts, cached outputs, and evaluation scripts are stored locally in the repository.
- The dataset path, model name, generation rounds, candidate counts, and repair settings are explicit in the code and run commands.
- The method writes intermediate debug files and can resume interrupted runs.
- All quantitative claims in the paper are tied either to reproduced evaluation outputs or explicitly labeled internal audit analysis.
- The main remaining source of variance is API latency and availability rather than algorithmic randomness.